

Threads in Java:

→

What is Threads:

A Thread is a lightweight sub-process that represent an independent path of execution within a program.

In simple words:

A Thread is a **small unit** of execution that allows a program to perform **multi task** concurrently.

for eg:

a music application can:

- play music,
- download,
- Update the UI,

using different threads.

→ Process vs Thread:

Process:

A process is a program that is currently running.

Eg:

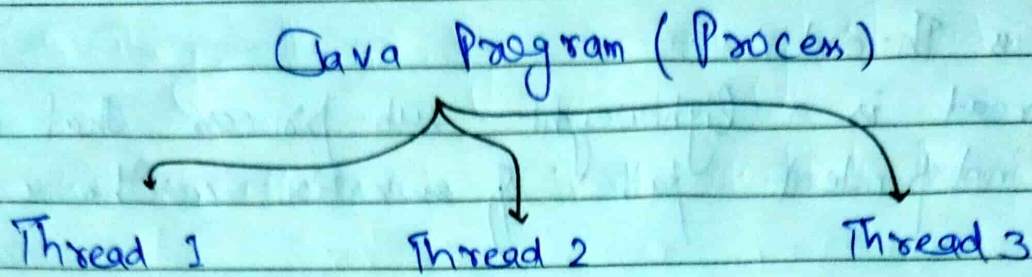
Chrome → Process

Vs code → Process

Game Program → Process

Thread:

A Thread is a smaller execution unit inside a process.



A single process can contain multiple threads.

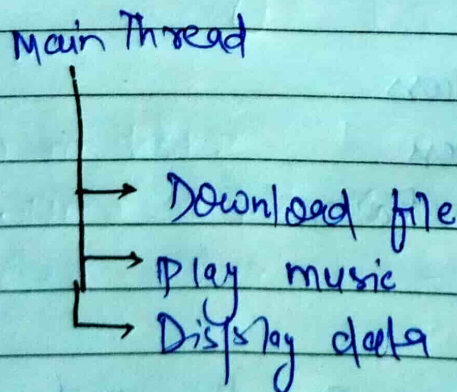
→ Why use threads?

Threads are used to perform multiple tasks concurrently.

Advantages:

1. Multitasking
2. Better CPU Utilization
3. Faster execution for suitable tasks
4. Improve responsiveness
5. Resource sharing between threads
6. Useful for background tasks

Eg:



→ What is Multithreading?

Multithreading is the process of **executing multiple threads** concurrently within a program.

Eg;

Program

```

├─ Thread 1 → Task A
├─ Thread 2 → Task B
└─ Thread 3 → Task C
    
```

→ Main Thread:

When a java program starts, JVM automatically creates a thread called the **main thread**.

Eg;

```

class test {
    public static void main(String[] args) {
        sout("Hello World");
    }
}
    
```

The main() method executes inside the **main thread**.

→ Creating thread in java:

There are two commonly ways:

Method 1: Extending thread class

Method 2: Implementing Runnable interface.

→ Creating Thread by Extending Thread:

We create a class that extends Thread.

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        System.out.println("Thread is running");
```

```
    }
```

```
}
```

```
class Test {
```

```
    public void
```

```
    MyThread t1 = new MyThread();
```

```
    t1.start();
```

```
}
```

```
}
```

Output:

Thread is running

→ run() method:

The run() method contains the task that the thread perform.

```

class MyThread extends Thread {
    public void run() {
        sout("task is running");
    }
}
    
```

But we normally **do not call run()** we want a new thread.

→ **Start() method:**
 Start() is used to start a new thread.

t.start();

When start() is called:

start()



JVM creates a new thread



Thread enters execution



run() is called

→ Creating a class using Runnable:

```
class MyTask implements Runnable {
```

```
    public void run() {
```

```
        System.out.println("Thread is running");
```

```
    }  
}
```

```
class Test {
```

```
    public static void main(String[] args) {
```

```
        MyTask task = new MyTask();
```

```
        Thread t1 = new Thread(task);
```

```
        t1.start();  
    }  
}
```

Flow:

MyTask
↓
implements Runnable

↓
run()

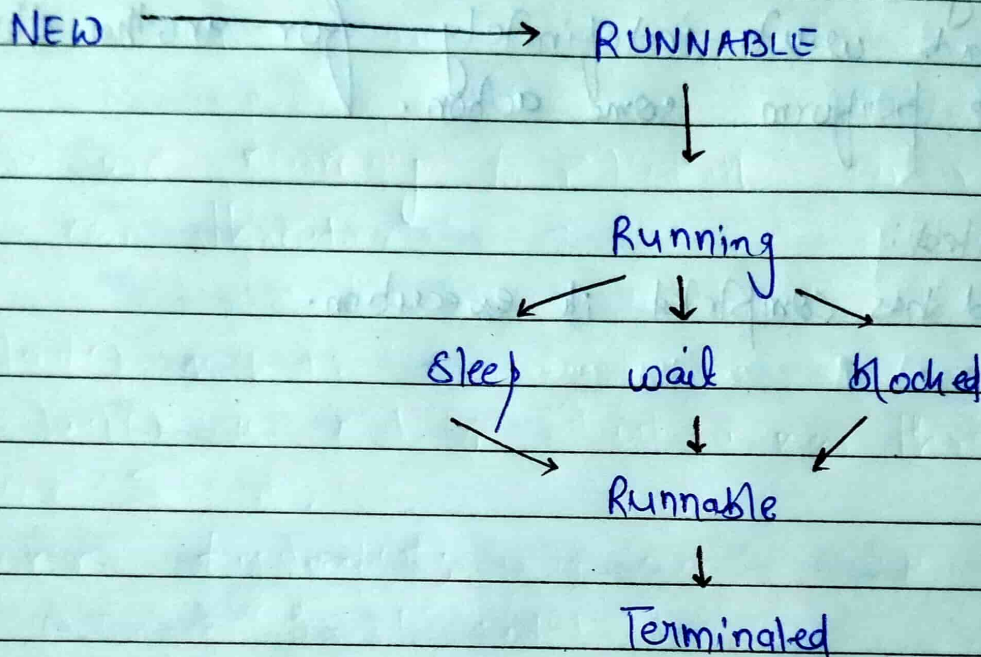
↓
Thread object

↓
start()

↓
New Thread

→ Thread life cycle:

1 Thread goes through different states during its life time.



• NEW:

Thread object is created but `start()` has not been called.

Thread t = new Thread();

• RUNNABLE:

t.start();

The thread become eligible to execution.

→ **Blocked:**
Thread is waiting to acquire a monitor lock.
This commonly occurs with synchronized code.

→ **Waiting:**
Thread waits indefinitely for another thread to perform some action.

→ **Terminated:**
Thread has completed its execution.

→ Race condition:

A race condition occurs when the result of a program depends on the timing / order in which multiple threads access shared data.

Eg:

Count = 0

Then

Thread 1 → read count

Thread 2 → read count

Thread 1 → count + 1

Thread 2 → count + 1

Both may read the same old value, producing an incorrect final result.

→ Thread Communication:

Thread can communicate with each other using:

wait()

notify()

notifyAll()

These methods are associated with an object's monitor and are used for coordination b/w threads.

Thread A



wait()



waits



Thread B



notify



Thread A can continue

